WiFeS data download

自动下载WiFeS观测数据的脚本，原理为使用request自动爬取相应的frame链接，生成对应的wget链接，然后批量写入.sh可执行文件，后运行.sh脚本下载。

用法可以参见WiFeS-dataDawnload/data_download.ipynb。science数据和calibration数据是分开下载的。

脚本依赖:

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin
from pathlib import Path
import subprocess
from concurrent.futures import ThreadPoolExecutor
import os
```

WiFeS data download：科学数据下载例子

```
username="yshipku@gmail.com"
password="N^3BPcXX^YwtNm"

root_path  = 'data/'
obj_name   = 'ESO245-G007'
semester   = 'S-252/'
Program_ID = '2523031'
grisms=["U7000", "R7000"]

d_sci = WiFeSSciDownload(
    username=username,
    password=password,
    semester=semester,
    PID=Program_ID,
    obj_name=obj_name,
    data_download_path=root_path,
    root_url="https://muggy.anu.edu.au/aocsarchive/content/"
)

d_sci.run(download=False, max_workers=8)
```

数据下载账户名，密码

root_path：可变，字符串，数据下载目录

obj_name：可变，字符串，需要下载的源名字

Semester：可变，字符串，观测季度的总目录，需要检查后更改

Program_ID：可变，字符串，proposal ID

Grisms：可变，字符串列表，需要下载数据的grism列表

root_url：WiFeS data archive的链接，一般不用更改

download：可变，bool。True：自动下载，False：只生成.sh可执行脚本，不下载数据。

max_workers：可变，int，最大并行数量。

WiFeS data download：calibration数据下载例子

```python
dates = []
sci_info_path = glob.glob(root_path + '/sci/*.txt')[0]
if sci_info_path:
    with open(sci_info_path, "r") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

            parts = line.split()
            date = parts[-1]
            dates.append(date)

    print(dates)
else:
    print("No file exist!")

for d in dates:
    obs_month = 'UT-' + d[:-3]
    obs_date = d.split('-')[0][2:] + d.split('-')[1] + d.split('-')[2]
    cal_data_path = root_path + obs_date + '/'

    d_cal = WiFeSCalibDownload(
        username=username,
        password=password,
        obs_month=obs_month,
        grisms=grisms,
        obs_date=obs_date,
        data_download_path=cal_data_path,
        root_url="https://muggy.anu.edu.au/aocsarchive/content/Common/"
    )

    d_cal.run(download=False, max_workers=8)
```

Calibration数据依赖已下载/生成的science目录来读取日期信息。

其余参数设置与上页中下载science数据的配置类似，基本不需要额外修改

download：可变，bool。True：自动下载，False：只生成.sh可执行脚本，不下载数据。

max_workers：可变，int，最大并行数量。

PyWiFeS pipeline：安装

我们工作站的配置是在Anaconda的python 2.7环境中放一份软件包，
在实际工作目录放一份。需要在.bashrc中配置环境变量如下：

```
export PYTHONPATH="/opt/anaconda3/envs/pywifes/lib/python2.7/site-packages/PyWiFeS-pipeline/src"
```

之后在工作目录中处理数据

PyWiFeS pipeline：step by step

1.进入py2环境, [conda activate pywifes]
2.把所有raw data放入PyWiFeS-pipeline/reduction_scripts/data/目录下
3.进入PyWiFeS-pipeline/reduction_scripts/extra_tools/
4.[python generate_metadata_script.py] + 数据路径（产生save_blue(red)_metadata.py文件）分类文件。
5. 需要检查save_blue(red)_metadata.py文件中对raw data分类是否正确。
6.运行4中产生的文件 [python save_blue(red)_metadata.py]， 产生pkl文件
7.返回上级目录：PyWiFeS-pipeline/reduction_scripts/
8.处理数据 [python reduce_blue(red)_data.py wifesB(R)_xxxxx.pkl]，需要更改使用的波段，使用对应波段的code，把其他注释掉。中途出现的图需要关掉。
9.处理过程中产生的文件在 reduc_b(r)/目录下，最终生成的datacube是文件名为：xxxxp11.fits，

PyWiFeS pipeline：注意事项

ps1: 在处理红端(R&I)的数据时，需要去改wifes_calib.py (path = /opt/anaconda3/envs/py27/lib/python2.7/site-packages/PyWiFeS-pipeline/src/wifes_calib.py) 在大约700行，取消注释掉的两行。

```
        numpy.min(full_x),
        1.000001*numpy.max(full_x),
        0.0001*(numpy.max(full_x)-numpy.min(full_x)))

    # ##############modifiled by us---- red band using##################
    # if numpy.max(final_x) > numpy.max(smooth_x):
    #     final_x[-1] = smooth_x[-1]
    # ##############modifiled by us##################

    final_y = this_f(final_x)



    # print('&&&&&&&&-----&&&&&&&&')
```

ps2: N&S mode，需要在reduce_red_script.py中设置step ‘slitlet_mef’, ‘cosmic_rays’, ‘sky_sub’ 内的args：ns 为True，来正常处理N&S观测模式的数据

```
NOD+SHUFFLE INSTRUCTIONS

In principle nod+shuffle data works exactly the same way as a separate sky observation except that the data are
initially contained in the same file.  Thus if you set 'ns':True for the MEF, cosmic rays, and sky subtraction
steps, it will work correctly.

The only difficulty is if you have *some* science observations in N+S and others not.  I plan to make the script
robust enough to tell the difference in the future, but for now you should run the reduction script on science
data of all the same format (note you can rewrite the script to not use N+S on science observations but not
standard star observations).
```

PyWiFeS pipeline：注意事项

Ps3: 更新后script无法正确分类标准星文件，需要在对应的save_blue(red)_metadata.py中手动添加，具体为在该.py文件中的std分类下添加当天观测的标准星文件名，格式如下：

```
{'sci'  : ['OBK-124416-WiFeS-Blue-UT20250627T082218-8',
           'OBK-124416-WiFeS-Blue-UT20250627T082458-1',],
 'name' : ['HD200654'],
 'type' : ['flux', 'telluric']},
```

{'sci' : ['OBK-124416-WiFeS-Blue-UT20250627T082218-8',
‘OBK-124416-WiFeS-Blue-UT20250627T082458-1',],
'name' : ['HD200654'],
‘type' : ['flux', 'telluric']},

需更改文件名

WiFeS post-process

包含的所有步骤

```
"FileSetup",
"SkySubstraction",
"CombineEpoch",
"GenerateBroadbandImages",
"GrismFluxNormalize",
"CombineGrism",
"UpdateWCS",
"DeRedden",
"CombineMultiFoVs",
```

重新组织文件
减blank sky，（如果需要）
combine多次曝光，单个grsim
根据IFS datacube生成伪宽波段图像
与skymapper image对比，生成flux normalize 系数的列表
拼接多个grism的光谱
更新header中的WCS信息，并旋转图像至上北左东方向
修正银河系消光
拼接多个pointing，（如果需要）

脚本依赖

```
import os
import glob
import yaml
import pyphot
import csv
import logging
import numpy as np
import astropy as AP
import astropy.units as u
import warnings
```

```
from astropy import wcs
from astropy.table import Table
from astropy.io import ascii,fits
from astropy.coordinates import SkyCoord
from dataclasses import dataclass
from abc import ABC, abstractmethod
from tqdm import tqdm
from datetime import datetime
from pathlib import Path
from multiprocessing import cpu_count
from pathos.multiprocessing import ProcessingPool as Pool
from reproject.mosaicking import find_optimal_celestial_wcs
from reproject import reproject_adaptive
```

WiFeS post-process：

```
obj_name  = 'UGC00695'
pointing  = '01:07:46.93 01:03:54.1'
root_path = '/Volumes/Malu/Data_Reduction/WiFeS/UGC00695/BR3000/'
```

```
pipeline = IFUPipeline()

log_info = pipeline.setup_advanced_logging(
    log_dir = root_path + 'logs/',
    prefix = 'UGC00695',
    log_level = logging.INFO,
    console_output=True,              # 同时显示在控制台
    separate_error_log=True,           # 单独保存错误日志
    max_bytes=10*1024*1024,            # 单文件最大10MB
    backup_count=5                     # 保留5个备份
)


# pipeline.add_step(FileSetup())
# pipeline.add_step(SkySubstraction())
# pipeline.add_step(CombineEpoch(n_jobs=2))
# pipeline.add_step(GenerateBroadbandImages(filter_name=None))
# pipeline.add_step(GrismFluxNormalize(filter_name=None))
pipeline.add_step(CombineGrism(n_jobs=4))
pipeline.add_step(UpdateWCS())
pipeline.add_step(DeRedden())
pipeline.add_step(GenerateBroadbandImages(filter_name=None))
# pipeline.add_step(CombineMultiFoVs())

pipeline.run(
    obj_name  = obj_name,
    root_path = root_path,
    pointing  = pointing,
)
```

obj_name：可变，str，需要处理的源的名称
Pointing：可变，str，观测时的坐标
root_path：可变，str，程序运行时的目录。

需要手动创建目录：
root_path/pipeline_frames/，将所有需要处理的数据放在其中，（包含science，sky frame）
root_path/broadband_image/将archive中的宽波段图像放在其中。并使用DS9在图像中选择一块矩形空白sky作为背景，保存为blank_sky.reg文件，.reg文件中的坐标格式为fk5。

可以参考其中UGCA193的文件树，
reduced_file/和logs/是可以自动创建的

WiFeS post-process：

```
obj_name  = 'UGC00695'
pointing  = '01:07:46.93 01:03:54.1'
root_path = '/Volumes/Malu/Data_Reduction/WiFeS/UGC00695/BR3000/'
```

```
pipeline = IFUPipeline()

log_info = pipeline.setup_advanced_logging(
    log_dir = root_path + 'logs/',
    prefix = 'UGC00695',
    log_level = logging.INFO,
    console_output=True,              # 同时显示在控制台
    separate_error_log=True,          # 单独保存错误日志
    max_bytes=10*1024*1024,           # 单文件最大10MB
    backup_count=5                    # 保留5个备份
)


# pipeline.add_step(FileSetup())
# pipeline.add_step(SkySubstraction())
# pipeline.add_step(CombineEpoch(n_jobs=2))
# pipeline.add_step(GenerateBroadbandImages(filter_name=None))
# pipeline.add_step(GrismFluxNormalize(filter_name=None))
pipeline.add_step(CombineGrism(n_jobs=4))
pipeline.add_step(UpdateWCS())
pipeline.add_step(DeRedden())
pipeline.add_step(GenerateBroadbandImages(filter_name=None))
# pipeline.add_step(CombineMultiFoVs())

pipeline.run(
    obj_name  = obj_name,
    root_path = root_path,
    pointing  = pointing,
)
```

log_dir：可变，str，存放处理日志的目录，通常不需要更改
Prefix：可变，str，处理日志的前缀，我通常设置为源的名称。
其余log_info中的变量都不是很重要，可以不变。

初始化程序为开始的pipeline = IFUPipeline()
后续需要添加的步骤可以按图中所示使用pipeline.add_step(step_name())来添加，
其中部分步骤的参数：
n_jobs：可变，int，最大并行数量
filter_name：可变，list，包含filter名字的字符串列表，默认为skymapper的filter

运行的code可以参考test.ipynb内“pipeline test” section中的“single object”与“multiple objects”部分。